Final Year
COMP63038 Clean Coding & Concurrent Programming

LUDO WITH A TWIST

Assignment 2: refactoring a single-process simulation into a multi-user, multi-tier client–server application

Sharini Nirmalan
CB014426

October 9, 2026

Contents

1 Introduction **1**

2 Starting Point: the Assignment 1 Solution **1**
2.1 What Assignment 1 delivered . . . 1
2.2 What it did not have . . . 1

3 Refactoring: the Issues Faced **2**

4 Solutions **3**
4.1 Architecture: three processes around an unchanged core . . . 3
4.2 Solution to each issue . . . 4
4.3 Allocation of Clean Architecture components to the tiers . . . 4
4.4 One turn across the tiers . . . 6
4.5 The concurrency design . . . 6

5 Technology of the Solution **10**

6 Implementation and Implementation Strategy **11**
6.1 Strategy . . . 11
6.2 The graphical user interface . . . 11
6.3 Evidence that it works . . . 14

7 Discussion: Changes from Assignment 1 and Design Trade-offs **16**
7.1 What changed from the Semester 1 assignment . . . 16
7.2 Design patterns and SOLID after the refactoring . . . 16
7.3 Design trade-offs . . . 17
7.4 Critical analysis of the concurrency mechanisms . . . 18
7.5 Clashes between patterns and principles . . . 20
7.6 Limitations and what could be done better . . . 21

8 Test Cases and their Updates **21**
8.1 Test suite . . . 21
8.2 How the tests were updated . . . 22
8.3 Testing concurrency deterministically . . . 23

9 Conclusion **24**

References **25**

A Running the System **26**

B HTTP API (embedded Tomcat, port 8081) **26**

List of Figures

1 Deployment: client, game server and database server as three independently launched processes . . . 3
2 Clean Architecture layers of LUDO-T and the tier each layer executes in . . . 6
3 Sequence diagram . . . 7
4 Threads of the game server: producers, bounded queue, consumers, room lock and persistence writer . . . 7
5 `RequestQueueProcessor.enqueue()`: the timed, blocking offer into the bounded queue . . . 8
6 `GameRoom.handleRollDice()`: the whole action runs under the room lock, released in `finally` (lines 276–278 omitted) . . . 9
7 `GameEventsServlet`: asynchronous long-poll with an exactly-once response . . . 10
8 Mid-game: Blue to roll, the mystery cell on the track, live scorecards on the right . 12
9 Lobby waiting for four players (left) and the end of a match (right) . . . 12
10 Chat and leaderboard tabs; the leaderboard is read from the database tier through the server . . . 12
11 Two Swing clients in one match captured at the same instant . . . 13
12 Swing client (Blue) in a match whose other three players play over HTTP . . . 13
13 Console output of the H2 database server (Tier 3) and the game server (Tier 2) . . 14
14 Stress clients against the live deployment, followed by the server's own queue counters 15
15 `test.bat`: 280 tests, 0 failures . . . 22
16 `RequestQueueProcessorTest.backPressureWhenFull()`: forcing the full-buffer path . . . 23

List of Tables

1 Issues faced during the refactoring . . . 2
2 How each issue was solved . . . 4
3 Allocation of components to tiers and Clean Architecture layers . . . 5
4 Technology stack and the reason for each choice . . . 10
5 Stress test: four automatic clients, ten seconds, heartbeat storm plus a full match . 14
6 JMeter results (3,021 samples, 0 errors, about 1,500 samples per second) . . . 15
7 Assignment 1 compared with Assignment 2 . . . 16
8 Design patterns and SOLID principles: preserved, changed or added . . . 17
9 Concurrency mechanisms used and rejected . . . 19
10 Test classes for Assignment 2 . . . 21
11 Launch scripts . . . 26
12 HTTP endpoints . . . 26

1 Introduction

Assignment 1 produced *LUDO-T*, a console simulation of Ludo with a set of "twist" rules in which four AI strategies play against each other inside one Java process. Assignment 2 asks for the same game to become a **multi-user, multi-tier, client–server application**: people on different computers play through a graphical interface, a server owns the rules and copes with many fast automatic clients at once, and a database keeps results between matches — while the Clean Architecture, design patterns and SOLID properties of the first submission are preserved or their removal justified.

The report follows the path the work actually took:

1. **Where we started** — the Assignment 1 design and what it did not offer (Section 2).
2. **Refactoring issues** — the problems that had to be solved, both those foreseen and those that appeared during the work (Section 3).
3. **Solutions** — the architecture and the mechanism chosen for each problem, including the allocation of Clean Architecture components to tiers (Section 4).
4. **Technology** — what the solution is built with and why (Section 5).
5. **Implementation and strategy** — how it was built, and evidence that it works (Section 6).
6. **Discussion** — what changed from Assignment 1, the design trade-offs, and a critical analysis of the concurrency mechanisms available and chosen (Section 7).
7. **Test cases and their updates** (Section 8).

2 Starting Point: the Assignment 1 Solution

2.1 What Assignment 1 delivered

LUDO-T extends standard Ludo with captures that grant a bonus roll, impenetrable same-colour blockades, and a *mystery cell* that appears on the track, teleports the piece that lands on it and applies a four-round status effect (energised, sick or "briefing"). The Assignment 1 solution was a single-threaded console program, `ludo.Main`, in which `GameEngine` ran rounds until all four colours had finished.

- **Domain (entities):** `Board`, `Piece`, `Player`, `Dice`, `CoinToss`, `MysteryCell`, `MoveResult`, the enums `Color`, `PieceState`, `MysteryEffect`, `Direction`, and `BoardUtils`.
- **Use cases:** `GameEngine`, `TurnManager`, `MoveProcessor`, `MysteryManager`.
- **Patterns:** Strategy for the four AI players (`PlayerStrategy`) and the six mystery teleports (`TeleportEffect`); Singleton for `Dice`; Observer for output (`GameEventListener` with `ConsoleGameListener`); Null Object (`MysteryEffect.NONE`); GRASP Controller and Information Expert.
- **Tests:** 216 JUnit 5 tests (Mockito for collaborators), all passing.

2.2 What it did not have

Measured against the Assignment 2 brief, the starting point lacked everything outside the domain: there was no user interface other than printed text; one process did everything, so there were no tiers; nothing ran concurrently, so no class was thread-safe or needed to be; there was no persistence; and the only "players" were strategies that the engine called synchronously. Two design decisions from Assignment 1 nevertheless turned out to matter a great deal. The *Observer* seam meant the engine never printed anything itself, and the *Clean Architecture* dependency rule

meant the domain imported nothing from an outer layer. Both made it possible to put a network around the engine instead of rewriting it.

3 Refactoring: the Issues Faced

Table 1 lists the problems the refactoring had to solve. The first group was visible from the brief before any code was written; the second group only appeared while testing and executing the program.

Table 1: Issues faced during the refactoring

#	Issue	Why it is a problem
Foreseen from the brief		
I1	The engine is a closed loop	`GameEngine.runGame()` drives all four players itself. A networked game must wait for a human's click, so the turn flow has to be driven from outside, one request at a time.
I2	Rules hidden in the console loop	The three-sixes penalty and the "briefing" escape lived in private methods of `TurnManager.playTurn()`, so a server could not reuse them without copying them.
I3	Shared mutable state	`Board` and `Piece` mutate in place. Once several threads handle requests for the same room, every read and write of turn state becomes a race unless it is serialised.
I4	Floods of requests	The rubric asks for many fast automatic clients. One thread per request would create unbounded threads all fighting for the same room; requests that cannot be served at once must be queued.
I5	Two kinds of client	The GUI needs the server to *push* events; tools such as Postman and JMeter speak request/response HTTP, which cannot be pushed to.
I6	Swing threading rule	Swing may only be touched on the Event Dispatch Thread, but network messages arrive on a socket-reading thread.
I7	A real database tier	Results and the leaderboard must outlive a single match and a server restart, so the data needs its own tier: a separate database process that is created and reset with standard SQL.
I8	Keeping the architecture clean	Networking, HTTP and SQL must be added without the domain learning about any of them.
Found during implementation and testing		
I9	Mystery cell never appeared online	The engine starts the spawn countdown when told a piece entered the board; the networked room never told it, and effects were never decremented at round end.
I10	Deadlock when closing a client	Closing a `BufferedReader` from another thread while its owner is blocked in `readLine()` hung the JVM (found with `jstack`).
I11	Dropped requests	A first queue used a non-blocking `offer()`: a 10,000-request flood dropped 1,137 requests.

#	Issue	Why it is a problem
I13	Overflow players stranded	JMeter seated 20 players in one room name. Every player beyond the fourth was put in a brand-new room of their own, so those matches never started.
I14	Silent requests	A roll sent before a match started got no answer at all, so each HTTP request waited out its full 1.5 s timeout: the JMeter roll sampler averaged 1,509 ms.
I16	Invisible roll button	Under the Windows look-and-feel a `JButton` ignores `setBackground()`; the white "ROLL DICE" label was drawn on a white face, exactly when it was the player's turn. Found only by looking at the running client.
I17	Highlight one turn behind	The state snapshot was broadcast before the turn advanced, so every client highlighted the previous player's scorecard.

4 Solutions

4.1 Architecture: three processes around an unchanged core

Figure 1 shows the result. Tier 1 is a Swing client per player. Tier 2 is the game server: a TCP listener for the Swing clients and an embedded Apache Tomcat for HTTP clients, both feeding *one* bounded request queue whose worker threads run the Assignment 1 engine inside per-room locks. Tier 3 is an H2 database server reached over JDBC. Each tier is started by its own batch file and runs as its own operating-system process.

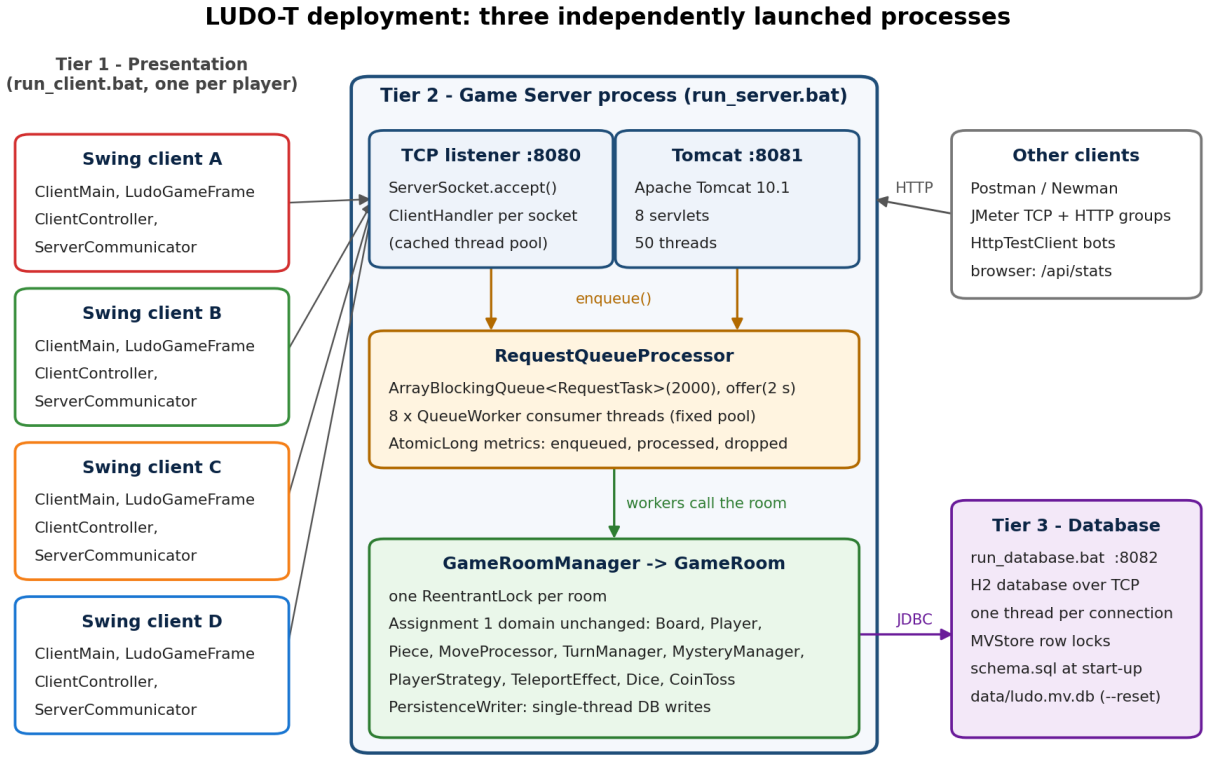


Figure 1: Deployment: client, game server and database server as three independently launched processes

4.2 Solution to each issue

Table 2: How each issue was solved

Issue	Solution
I1	A new use-case class, `GameRoom`, holds one match and exposes the turn as separate actions (`joinPlayer`, `handleRollDice`, `handleMovePiece`, `removePlayer`). It reuses `Board`, `MoveProcessor`, `MysteryManager` and `TurnManager` unchanged.
I2	The two rules were promoted to public `TurnManager` methods used by both the console loop and the room, with their own tests (`TurnManagerRulesTest`).
I3	One `ReentrantLock` per room; every action is one critical section with `unlock()` in `finally`. Different rooms never contend (Section 4.5).
I4	Producer–consumer: socket and HTTP threads only parse and enqueue a `RequestTask` into an `ArrayBlockingQueue` of 2,000; eight `QueueWorker` threads consume (Section 4.5).
I5	Two interface adapters behind one port, `ClientSession`: `ClientHandler` (socket, push) and `HttpClientSession` (HTTP). HTTP clients receive events by *long-polling* an asynchronous servlet.
I6	One listener thread per client reads the socket and hands every message to the EDT with `SwingUtilities.invokeLater()`.
I7	An H2 database server process; `schema.sql` is applied at start-up and `-reset` restores the empty initial state. The game server uses a JDBC connection pool behind `PersistenceGateway`.
I8	Two ports at the use-case boundary — `ClientSession` and `PersistenceGateway` — plus the existing `GameEventListener`. The domain packages import nothing from the outer packages.
I9	The room calls `notifyFirstPieceOnBoard()` after a move and decrements effects in `endRound()`; a test asserts that a full match produces mystery events.
I10	Close the socket first (which makes `readLine()` throw and release its lock) and never close a reader from a foreign thread — applied in all three socket closers.
I11	A timed blocking `offer(task, 2 s)`: back-pressure instead of loss.
I13	A full room now spills into `<id>-2`, `<id>-3`, ..., so overflow players are seated together and their match starts when four have arrived.
I14	Rolling or moving before the start, after the end, or moving before rolling is answered with an explicit `ERROR`. The JMeter roll average fell from 1,509 ms to 0.7 ms.
I16	The button paints its own background (`setContentAreaFilled(false)`, `setOpaque(true)`).
I17	`advanceTurn()` broadcasts the snapshot again after choosing the next player; a regression test checks every client's latest snapshot names the new active player, and fails without the fix.

4.3 Allocation of Clean Architecture components to the tiers

Table 3 places every class in a tier and a Clean Architecture layer, and Figure 2 shows the layers. The two inner layers are the Assignment 1 code: they were not made network-aware, they were given a new outer layer.

Table 3: Allocation of components to tiers and Clean Architecture layers

Tier / process	Layer	Classes	Responsibility
Tier 1 client (`run_client.bat`)	Frameworks & drivers	`ClientMain`, `LudoGameFrame`, `LudoBoardCanvas`, `ServerCommunicator`	Swing window, Graphics2D board, animations, log, chat and leaderboard tabs; socket listener thread.
Tier 1 client	Interface adapter	`ClientController`	GRASP Controller: clicks to messages, messages to view calls (on the EDT).
Shared	Interface adapter (DTO)	`GameMessage`, `MessageType`, `NetworkProtocol`	One text line per message.
Tier 2 game server (`run_server.bat`)	Frameworks & drivers	`ServerMain`, `LudoServer`, `ClientHandler`, `TomcatServerRunner`	TCP accept loop, one reader per socket, embedded Tomcat.
Tier 2 game server	Interface adapters	`ClientSession`, `RequestTask`, `RequestQueue Processor`, `HttpClientSession`, `LudoServlet` and seven servlets, `Json`	Producers and consumers of the queue; translation between HTTP/socket and the room API.
Tier 2 game server	Use cases	`GameRoomManager`, `GameRoom`, `TurnManager`, `MoveProcessor`, `MysteryManager`, strategies, teleports, `PersistenceGateway`	Room lifecycle, turn flow and every rule; the Assignment 1 engine is reused as-is.
Tier 2 game server	Entities	`Board`, `Piece`, `Player`, `Dice`, `CoinToss`, `MysteryCell`, `MoveResult`, enums	Unchanged domain model.
Tier 2 → Tier 3	Interface adapter	`JdbcPersistence Gateway`, `PlayerRecord`, `MatchRecord`, `SchemaScript`	SQL and the connection pool, behind the gateway port.
Tier 3 database (`run_database.bat`)	Frameworks & drivers	`DatabaseServer` (H2 TCP server), `schema.sql`	Separate process; creates or resets the schema; data in `data/ludo.mv.db`.
Test clients	–	`AutomatedTestClient`, `HttpTestClient`, `StressTestRunner`	Headless socket and HTTP players; metrics.

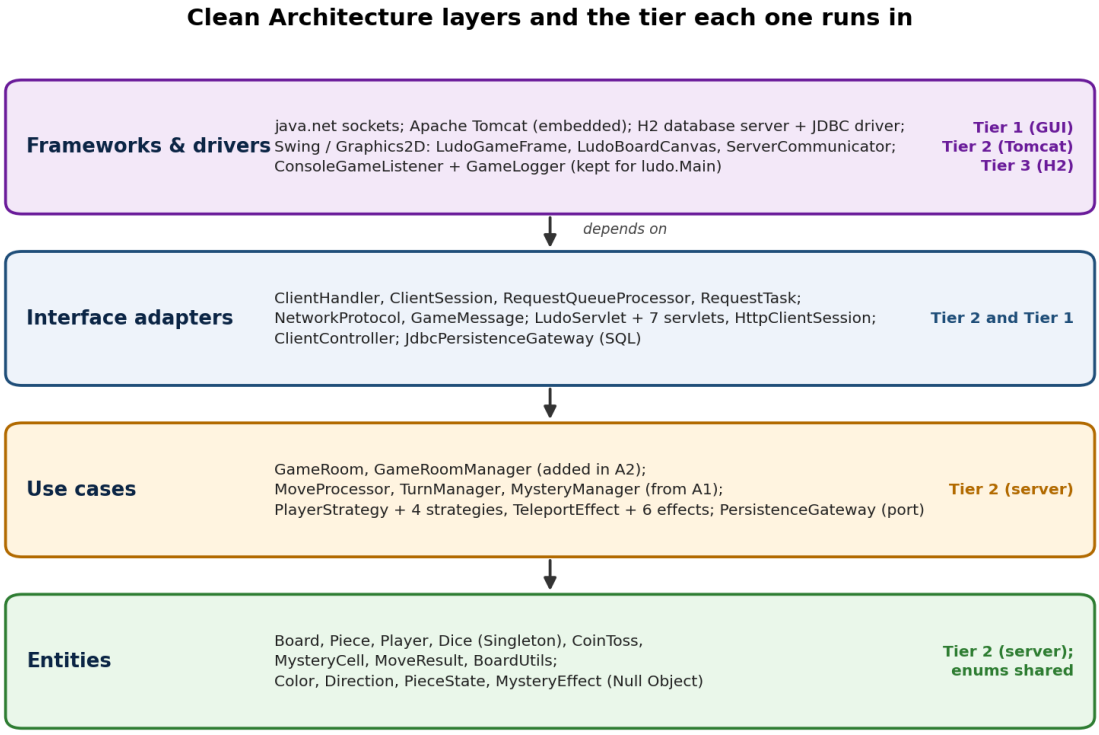


Figure 2: Clean Architecture layers of LUDO-T and the tier each layer executes in

4.4 One turn across the tiers

Figure 3 traces a roll and a move. Red's click becomes a `ROLL_DICE` line; the server's `ClientHandler` parses it and enqueues a `RequestTask`; a `QueueWorker` takes the task and calls `GameRoom.handleRollDice()` under the room lock; the outcome is broadcast to every session; each client's listener thread reads the line and hands it to the Swing thread. An HTTP request takes exactly the same path from the queue onwards — there is one way into the game logic.

4.5 The concurrency design

Figure 4 shows the threads inside the game server and how work moves between them.

The bounded queue. `RequestQueueProcessor` owns an `ArrayBlockingQueue<RequestTask>` of capacity 2,000 and a fixed pool of eight daemon worker threads. Producers — one `ClientHandler` thread per socket and Tomcat's request threads — only enqueue. When the buffer is full the producer waits up to two seconds; for a socket client that pause propagates back through TCP flow control, so the request is late rather than lost. A task that throws is counted and the worker carries on.

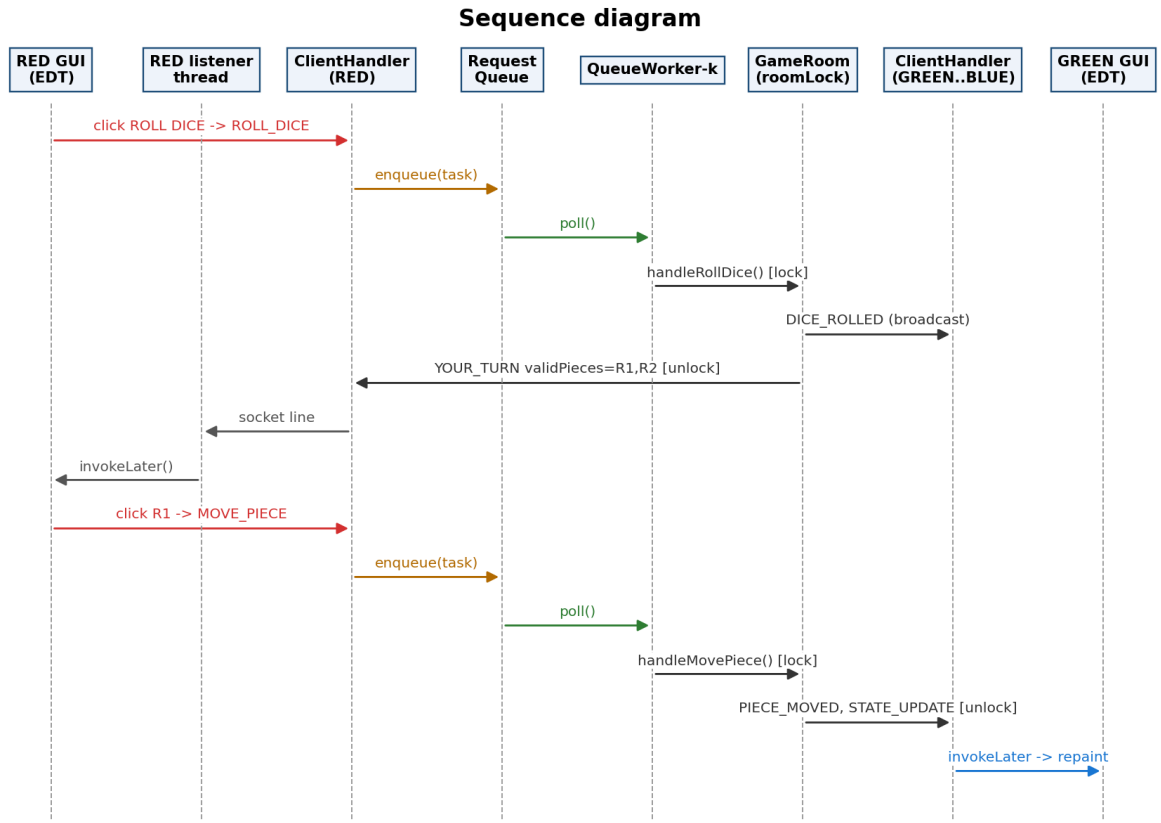


Figure 3: Sequence diagram

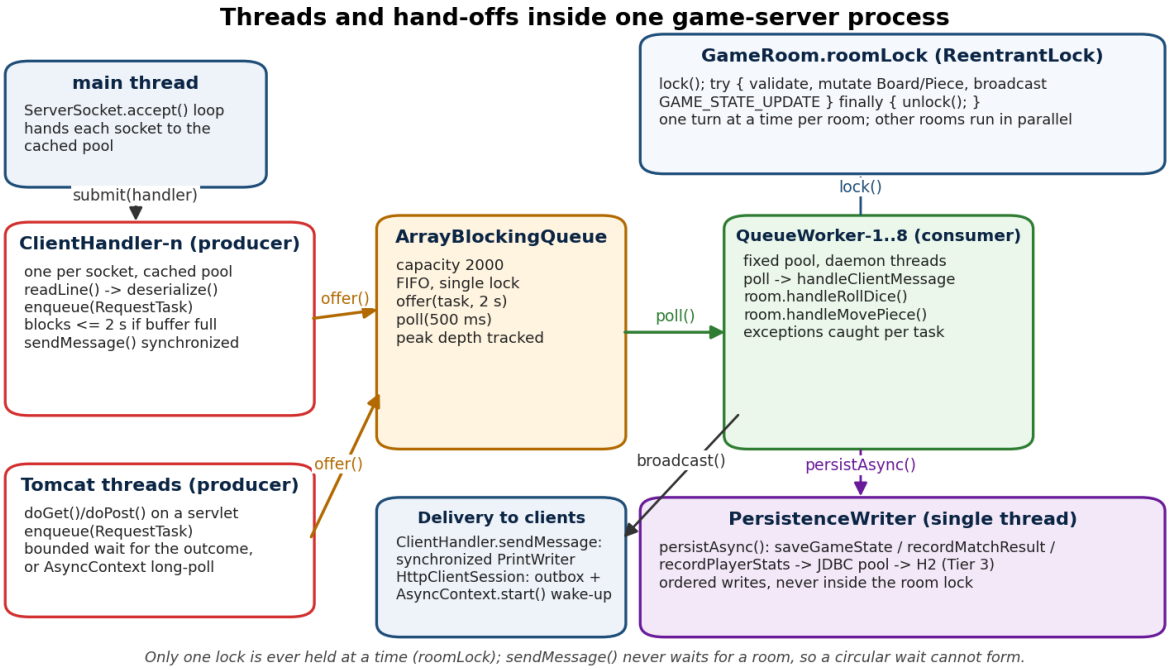


Figure 4: Threads of the game server: producers, bounded queue, consumers, room lock and persistence writer

```java
    /** Producer side; blocks up to ENQUEUE_TIMEOUT_MS when full. Returns false if rejected. */
    public boolean enqueue(RequestTask task) {
        return enqueue(task, ENQUEUE_TIMEOUT_MS);
    }

    public boolean enqueue(RequestTask task, long timeoutMs) {
        if (!isRunning) return false;
        totalEnqueuedTasks.incrementAndGet();
        boolean offered;
        try {
            offered = taskQueue.offer(task, timeoutMs, TimeUnit.MILLISECONDS);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            offered = false;
        }
        if (!offered) {
            totalDroppedTasks.incrementAndGet();
            System.err.println("[RequestQueueProcessor] Queue full for " + timeoutMs + " ms - rejected "
                    + task.getMessage().getType() + " from " + task.getSession().getPlayerName());
        } else {
            peakQueueDepth.accumulateAndGet(taskQueue.size(), Math::max);
        }
        return offered;
    }
```

Figure 5: `RequestQueueProcessor.enqueue()`: the timed, blocking offer into the bounded queue

One lock per room. Eight workers may pick up requests for the same room simultaneously. Every room action is one critical section, and the broadcast happens inside it, so all clients see updates in the order the room applied them.

```
246     public void handleRollDice(ClientSession client) {
247         roomLock.lock();
248         try {
249             if (rejectIfNotInPlay(client)) return;
250             Player activePlayer = getActivePlayer();
251             if (client.getAssignedColor() != activePlayer.getColor()) {
252                 client.sendMessage(new GameMessage(MessageType.ERROR, "SERVER", roomId)
253                         .put("message", "Not your turn!"));
254                 return;
255             }
256             if (waitingForPieceSelection) {
257                 client.sendMessage(new GameMessage(MessageType.ERROR, "SERVER", roomId)
258                         .put("message", "Please select a piece to move."));
259                 return;
260             }
261
262             currentDiceValue = dice.roll();
263             broadcast(new GameMessage(MessageType.DICE_ROLLED, "SERVER", roomId)
264                     .put("color", activePlayer.getColor().name())
265                     .put("diceValue", currentDiceValue));
266
267             // Rule T-8: three consecutive sixes forfeits the turn
268             if (turnManager.applyConsecutiveSixRule(activePlayer, board, currentDiceValue)) {
269                 broadcastGameState();
270                 advanceTurn();
271                 return;
272             }
273
274             // Rule T-14: briefing escape on consecutive threes
275             turnManager.trackBriefingEscape(activePlayer, board, currentDiceValue);
279             if (currentMovablePieceNames.isEmpty()) {
280                 systemMessage(activePlayer.getColor().getDisplayName() + " has no valid moves with roll " + currentDiceValue);
281                 broadcastGameState();
282                 if (currentDiceValue == BoardUtils.DICE_FACES) {
283                     promptActivePlayerTurn(); // a six always earns another roll
284                 } else {
285                     advanceTurn();
286                 }
287             } else {
288                 waitingForPieceSelection = true;
289                 String piecesCsv = String.join(",", currentMovablePieceNames);
290                 client.sendMessage(new GameMessage(MessageType.YOUR_TURN, "SERVER", roomId)
291                         .put("state", "SELECT_PIECE")
292                         .put("diceValue", currentDiceValue)
293                         .put("validPieces", piecesCsv));
294             }
295         } finally {
296             roomLock.unlock();
297         }
298     }
```

Figure 6: `GameRoom.handleRollDice()`: the whole action runs under the room lock, released in `finally` (lines 276–278 omitted)

The room lock is the only lock a worker ever holds while it waits for anything; `sendMessage()` never waits for a room, so there is no hold-and-wait chain and no circular wait — two of the four Coffman conditions for deadlock [8] cannot hold.

Persistence off the critical path. A database write is a network round trip. Doing it under the room lock would add that latency to a turn and let a slow database stall a game. `GameRoomManager` therefore owns a single-threaded executor, `PersistenceWriter`: the room hands it `saveGameState()` at round end and `recordMatchResult()` plus `recordPlayerStats()` at game over. One thread rather than a pool keeps the writes in issue order.

HTTP long-polling without blocking a thread. An HTTP player cannot be pushed to. `GET /api/game/events?waitMs=...` therefore uses Servlet asynchronous processing: if no event is waiting, the servlet calls `startAsync()` and returns, releasing the Tomcat thread. The session's one-shot listener fires when the next event is delivered. Because delivery happens on a `QueueWorker` that may hold a room lock, the listener does nothing except `AsyncContext.start()`, which hands the response write back to a Tomcat thread.

```
48          AsyncContext async = req.startAsync();
49          async.setTimeout(waitMs);
50          AtomicBoolean answered = new AtomicBoolean(false);
51
52          Runnable onEvent = () -> {
53              if (answered.compareAndSet(false, true)) {
54                  async.start(() -> respondAndComplete(async, session));
55              }
56          };
57          async.addListener(new AsyncListener() {
58              @Override
59              public void onTimeout(AsyncEvent event) {
60                  session.cancelNextEvent(onEvent);
61                  if (answered.compareAndSet(false, true)) {
62                      respondAndComplete(async, session); // already on a container thread
63                  }
64              }
65
66              @Override public void onComplete(AsyncEvent event) { session.cancelNextEvent(onEvent); }
67              @Override public void onError(AsyncEvent event) { session.cancelNextEvent(onEvent); }
68              @Override public void onStartAsync(AsyncEvent event) { }
69          });
70          session.onNextEvent(onEvent);
71      }
```

Figure 7: `GameEventsServlet`: asynchronous long-poll with an exactly-once response

Client side. A Swing client has two threads of its own: the EDT, which paints and handles clicks, and a daemon listener blocked in `readLine()`. The only hand-off between them is `invokeLater()` with immutable `GameMessage` objects, so they share no mutable state. Connecting happens on a short-lived thread so an unreachable server cannot freeze the window.

5 Technology of the Solution

Table 4: Technology stack and the reason for each choice

Concern	Technology	Why
Language and runtime	Java 21 (bundled JDK)	Same language as Assignment 1, records and pattern matching, `java.util.concurrent`; the JDK ships with the project so the demo machine needs nothing installed.
Desktop GUI	Swing / Java2D	Part of the JDK; Graphics2D draws the board directly from the server's state snapshot.
Client–server transport	`java.net` sockets with a one-line text protocol	Persistent connection, so the server can push; lines are readable in a terminal and testable from JMeter's TCP sampler.
HTTP tier	**Apache Tomcat 10.1 (embedded)**, Jakarta Servlet 6	The standard Java servlet container: real servlet lifecycle, a bounded NIO request pool, asynchronous servlets for long-polling. Embedded, so it starts inside the game-server process and shares its rooms and queue.
Concurrency	`java.util.concurrent`	Blocking queues, executors, explicit locks and conditions, atomics, concurrent collections.
Database	**H2 2.5 database server** (TCP mode)	A real SQL server in a single jar: runs as its own process, needs no installation, and is reached through standard JDBC. `schema.sql` creates the tables; `-reset` restores the initial state.

Concern	Technology	Why
Data access	JDBC with `JdbcConnectionPool`	Standard API; each call borrows its own connection, so threads never share a `Connection`.
Unit and integration tests	JUnit 5.10, Mockito 5.11	Same as Assignment 1, test-only.
Load and API testing	Apache JMeter 5.6, Postman / Newman	Named in the brief; both were run against the live server (Section 6.3).

6 Implementation and Implementation Strategy

6.1 Strategy

The work was done in increments, each one leaving the 216 Assignment 1 tests green, so the domain was protected by its regression suite throughout:

1. **Extract the turn.** Introduce `GameRoom` and promote the hidden rules (I1, I2); test the room with a recording test double before any network code exists.
2. **Wire protocol.** `GameMessage` and `NetworkProtocol`, round-trip tested.
3. **Server concurrency.** `ClientHandler` producers, the bounded queue, workers, room locks; deterministic concurrency tests.
4. **GUI.** Swing client against the running server; the EDT hand-off.
5. **Second transport.** HTTP servlets as further producers on the same queue.
6. **Database tier.** Gateway port, separate process, asynchronous writer.
7. **Automatic clients.** Socket and HTTP bots, stress runner, JMeter plan, Postman collection.
8. **Run every tool against the live system** — which exposed I13 and I14.

6.2 The graphical user interface

The console is replaced by a Swing client. Every use case is playable from it: joining, rolling, choosing a piece (only legal pieces are highlighted, computed by the server), captures, mystery teleports and effects, the three-sixes penalty, winning, chat, and the leaderboard that the database tier keeps between matches.

Immediate propagation between clients. The rubric asks that a change made by one client appears immediately on another. Figure 11 shows two client windows captured at the same instant: Blue's window highlights the piece the server offered, Red's shows "Waiting for BLUE", and both activity logs contain the same lines in the same order.

A live mixed match. Figure 12 was captured from the final build. Three players (Alice, Bob, Cara) joined and played their turns over HTTP; the fourth seat is the Swing client. Every HTTP roll appears in the GUI's activity log within the same second, the scorecards follow the board, and when the turn reaches Blue the client highlights Blue's card and enables *ROLL DICE*. Watching this run exposed defects I16 and I17, which no automated test had caught.

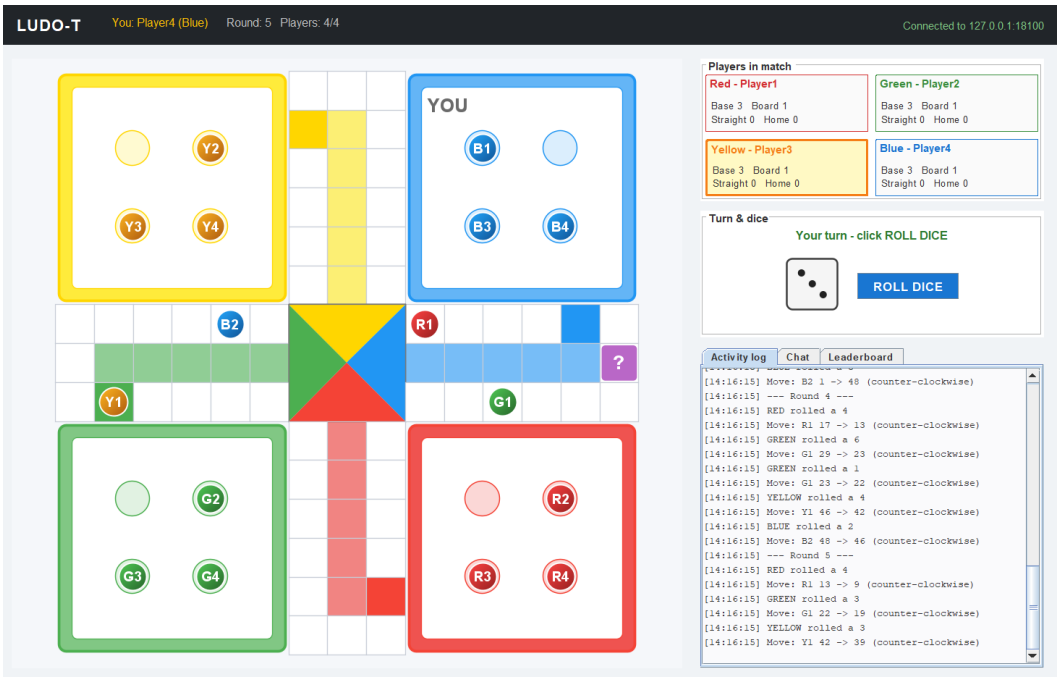


Figure 8: Mid-game: Blue to roll, the mystery cell on the track, live scorecards on the right

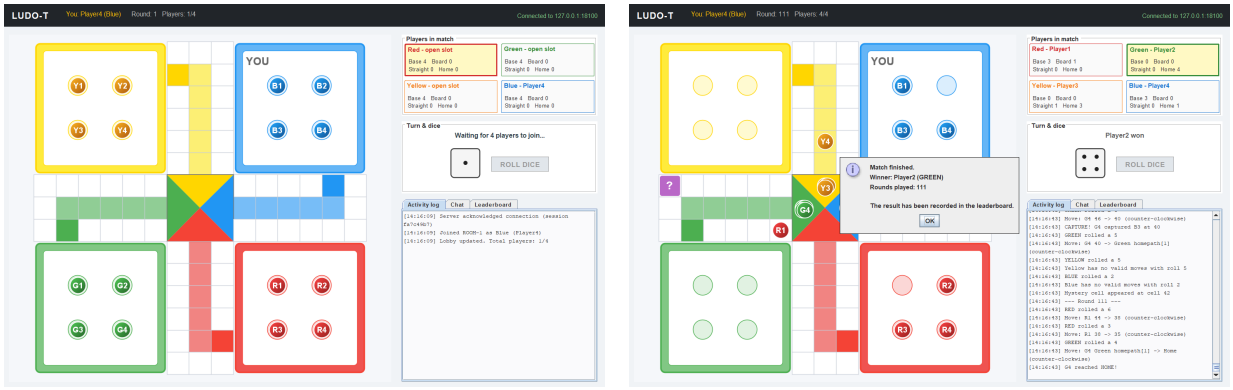


Figure 9: Lobby waiting for four players (left) and the end of a match (right)

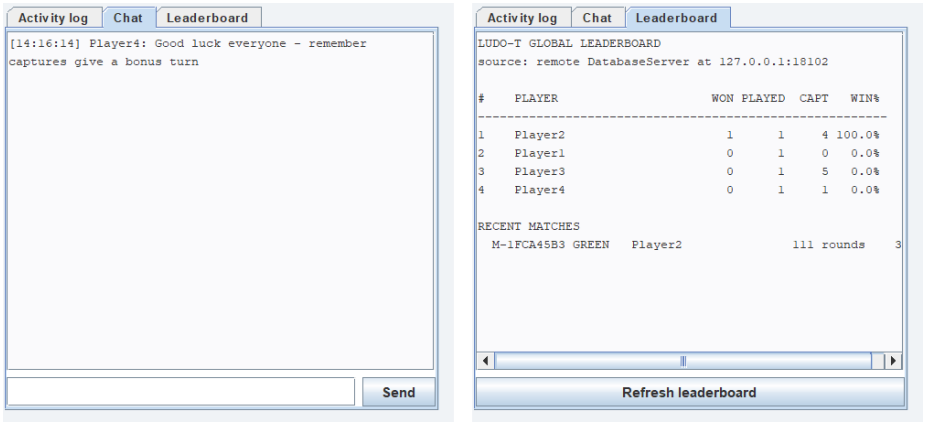


Figure 10: Chat and leaderboard tabs; the leaderboard is read from the database tier through the server

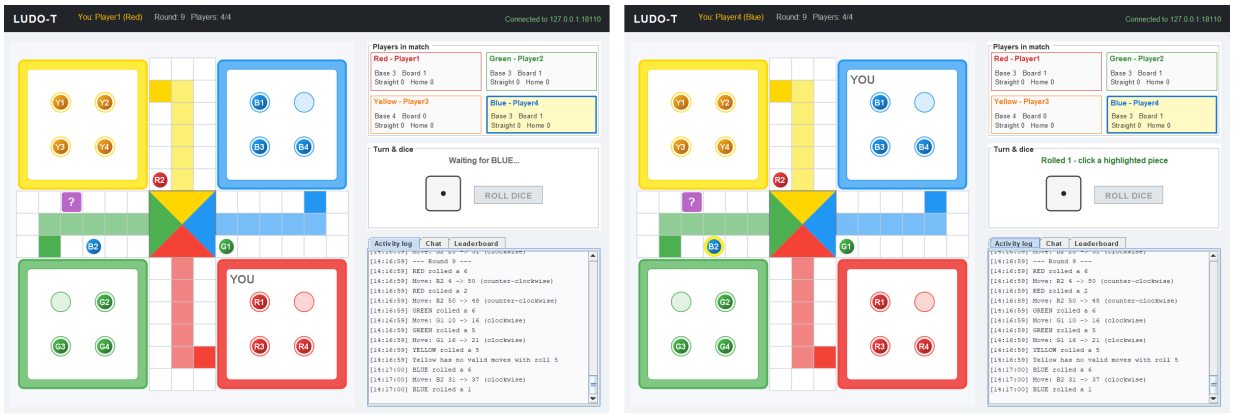


Figure 11: Two Swing clients in one match captured at the same instant

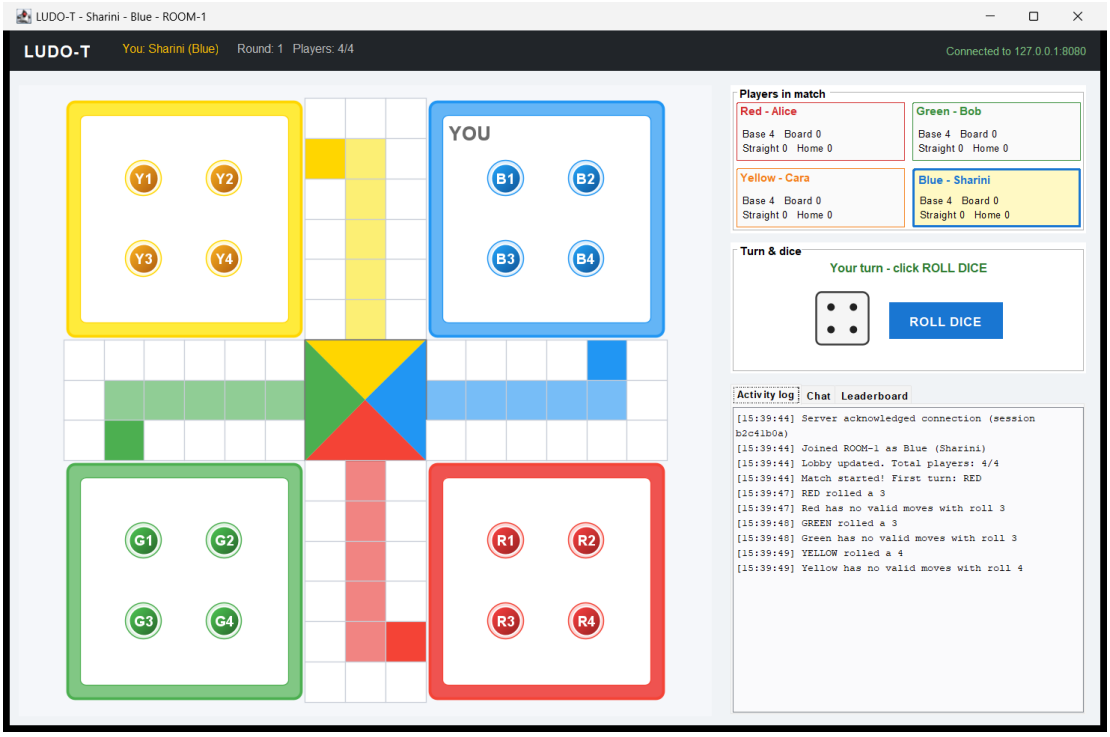


Figure 12: Swing client (Blue) in a match whose other three players play over HTTP

6.3 Evidence that it works

The tiers. Figure 13 shows the database server and the game server started as separate processes; the game server reports that it is connected to the H2 server and that Tomcat is serving HTTP.

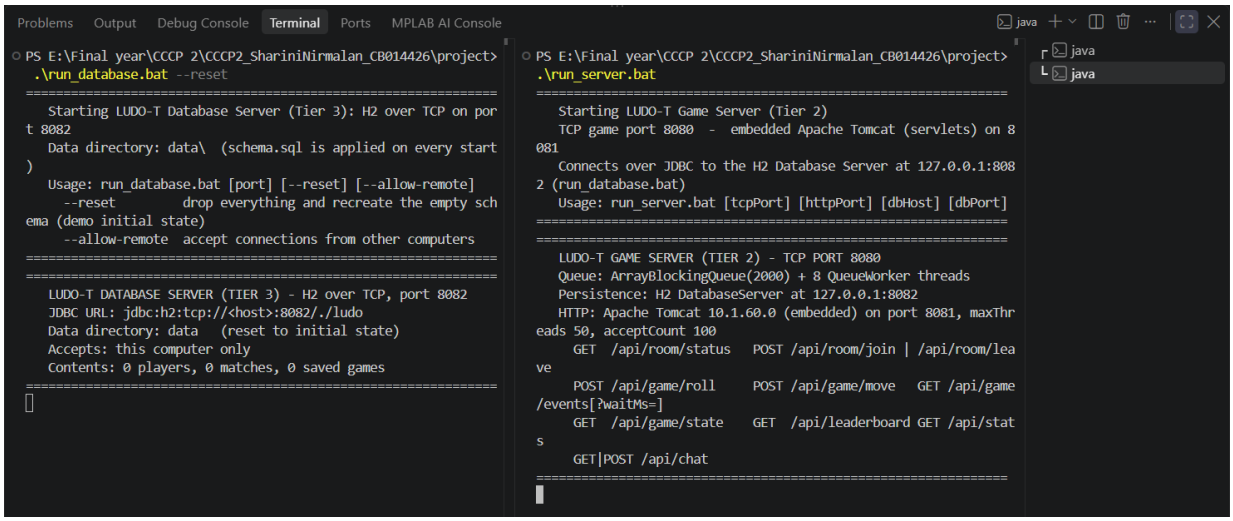


Figure 13: Console output of the H2 database server (Tier 3) and the game server (Tier 2)

Stress test with automatic clients. `StressTestRunner` connects four automatic socket clients that play a whole match as fast as the server allows, while a `ScheduledExecutorService` sends a `HEARTBEAT` from each of them every 50 ms. Table 5 shows a run against a private in-process server and a run against the live three-tier deployment.

Table 5: Stress test: four automatic clients, ten seconds, heartbeat storm plus a full match

Metric	Private server	Live three-tier deployment
Requests sent by the bots	1,355	1,853
Messages received by the bots	6,870	13,986
Server enqueued / processed / dropped	1,355 / 1,355 / 0	5,381 / 5,381 / 0 *
Peak queue depth	4 of 2,000	5 of 2,000
Average queue wait	0.068 ms	0.056 ms
Average processing time per task	0.258 ms	0.254 ms
Average heartbeat round trip (client-measured)	0.516 ms (792 samples)	0.468 ms (792 samples)
Match reached GAME_OVER	yes	yes (recorded in H2)

* Server counters are cumulative since start-up and include two earlier stress runs made against the same server.

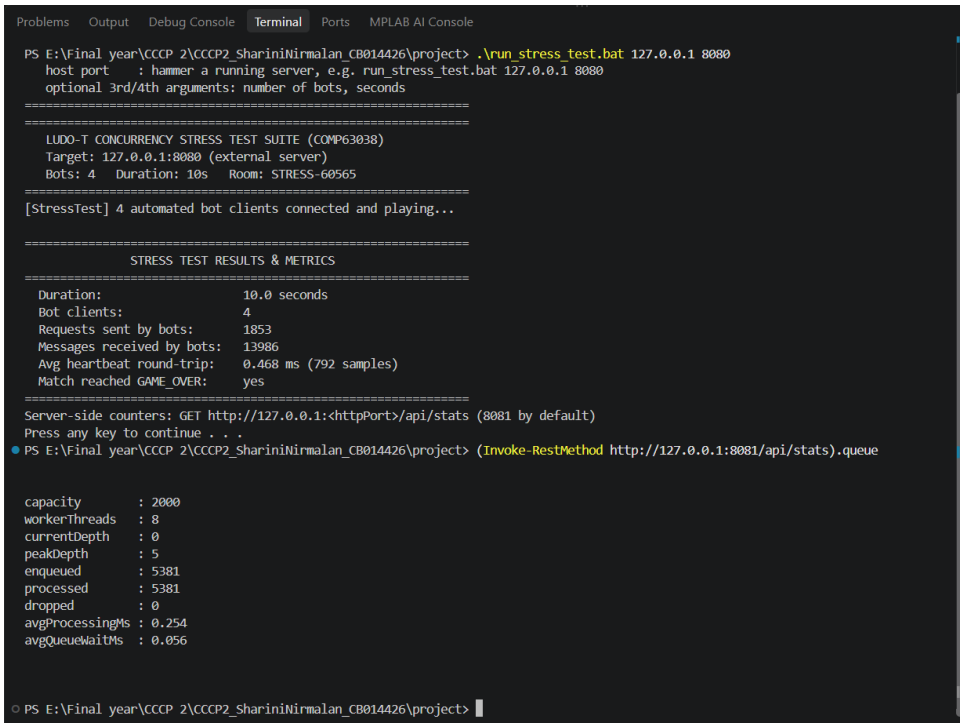


Figure 14: Stress clients against the live deployment, followed by the server's own queue counters

Four fast clients are far from the server's limit: the queue never held more than five tasks, so no producer ever had to wait. The other regime — a full buffer — is forced deterministically in a unit test (Section 8.3).

JMeter. `ludo_jmeter_test_plan.jmx` was run headless with Apache JMeter 5.6.3 against the live server. Twenty TCP threads send 50 heartbeats each while twenty HTTP threads each join a room once and then read the game state and roll 50 times; a tear-down group checks the server's `/api/stats`.

Table 6: JMeter results (3,021 samples, 0 errors, about 1,500 samples per second)

Sampler	Samples	Errors	Average	95th pct.	Max
HEARTBEAT over TCP	1,000	0	0.2 ms	1 ms	11 ms
POST /api/room/join	20	0	9.8 ms	58 ms	84 ms
GET /api/game/state	1,000	0	0.6 ms	1 ms	7 ms
POST /api/game/roll	1,000	0	0.7 ms	2 ms	8 ms
GET /api/stats (tear-down)	1	0	2.0 ms	2 ms	2 ms

The first JMeter run was not this clean, and it found two real defects. The roll sampler averaged 1,509 ms because the 20 players had been scattered into single-player rooms that never started (I13), and a roll in a room that has not started got no reply (I14). Twenty TCP samples also failed because the first line on a new connection is the server's `CONNECTED_ACK` greeting, which the plan's assertion did not expect — a defect in the test plan, fixed there. After the fixes the 20 JMeter players fill five rooms (`JMETER` to `JMETER-5`), every match starts, 989 rolls are rejected with `409 Not your turn` and 11 are accepted, and the whole run takes two seconds instead of 78.

Postman. `Ludo_Postman_Collection.json` (Postman v2.1) seats four HTTP players in one room, proves that an out-of-turn roll is rejected, rolls and moves, reads the state, long-polls the event stream, chats, reads the leaderboard, checks the error paths, and runs a small load folder. It was executed with Newman, Postman's command-line runner: **20 requests, 38 assertions, 0 failures** (`report/evidence/newman_results.txt`).

Long-polling. In the integration test, an HTTP client long-polls with a 10 s window and a second player joins 300 ms later; the poll returns with the join event as soon as it is delivered (the test asserts at least 250 ms and well under the 10 s timeout). Run by hand against the live server, a poll with `waitMs=5000` returned about 130 ms after the event that woke it.

7 Discussion: Changes from Assignment 1 and Design Trade-offs

7.1 What changed from the Semester 1 assignment

Table 7: Assignment 1 compared with Assignment 2

Aspect	Assignment 1	Assignment 2
Processes	One console program	Three: Swing client(s), game server, H2 database server
Players	Four AI strategies called by the engine	Humans in Swing, HTTP clients, automatic socket and HTTP bots
Turn flow	`GameEngine.runGame()` loop	`GameRoom` actions triggered by requests; `GameEngine` kept for `ludo.Main`
User interface	Console text via `ConsoleGameListener`	Swing/Java2D GUI; the console listener is kept for the CLI
Concurrency	None	Bounded queue, worker pool, per-room locks, async servlets, single-writer persistence, EDT hand-off
Persistence	None	H2 SQL database, separate process, JDBC pool, in-memory fallback
Domain classes	—	Unchanged; two `TurnManager` rules made public
New packages	—	`network`, `server`, `servlet`, `db`, `client`, `stress` (37 classes, 4,010 code lines)
Tests	216	280 (216 unchanged + 64 new)

7.2 Design patterns and SOLID after the refactoring

Table 8: Design patterns and SOLID principles: preserved, changed or added

Pattern / principle	Status in Assignment 2
Strategy (players)	Kept. Humans choose their own piece, so `choosePiece()` is unused in a networked match, but `MoveProcessor` still consults `preferBlockMove()`, the CLI still uses `choosePiece()`, and the four strategy test classes still run. Deleting a tested abstraction to save four small classes would be change for its own sake.
Strategy (teleports)	Unchanged; each outcome is now also broadcast as `MYSTERY_EVENT`.
Singleton (`Dice`)	Kept, with a new consequence: one dice object serves every room (Section 7.5).
Observer	The key enabler: the room's listener turns domain events into broadcasts, exactly as `ConsoleGameListener` turns them into text.
Null Object	Unchanged; every piece always serialises an effect name, which keeps the wire format simple.
GRASP Controller / Expert	Split by tier: `GameRoom` is the server-side controller, `ClientController` the client-side one; `Piece` is still the expert on its own movement.
New: Producer–Consumer	`RequestQueueProcessor`.
New: Ports and Adapters	`ClientSession` (socket and HTTP adapters); `PersistenceGateway` (JDBC adapter).
New: Template Method	Tomcat's `HttpServlet.service()` dispatches to the `doGet`/`doPost` overrides.
New: DTO	`GameMessage`, `PlayerRecord`, `MatchRecord` (Java records).
SRP	`GameRoom` is the new documented compromise (it coordinates seats, turn flow and broadcasting); the rules stayed in the engine classes, which is why they could be reused verbatim. Event delivery got its own servlet (`GameEventsServlet`) instead of a branch in the turn servlet.
OCP	New transports and storage were added by implementing ports, not by editing `GameRoom`.
LSP / ISP	`ClientSession` has only what a room calls; socket and HTTP sessions are interchangeable in every room test.
DIP	The server depends on `PersistenceGateway`; the concrete JDBC gateway is chosen in `ServerMain`. SQL exceptions are wrapped in `PersistenceException`, so no `java.sql` type crosses the port.

7.3 Design trade-offs

Thin client versus thick client.
Running the rules on every client would need consensus on whose dice roll is real and would let a modified client cheat. The server owns the rules; the client only draws. The cost is a round trip per action, measured at about half a millisecond on one machine.

Two transports versus one.
Supporting both sockets and HTTP doubles the adapters, but sockets give the GUI true push while HTTP lets standard tools test the system. Both feed one queue, so the game logic exists once.

Blocking versus dropping when the queue is full.
Blocking keeps every request, at the cost of a producer thread waiting up to two seconds. Dropping is simpler but breaks "process every request".

Embedded versus standalone Tomcat.
A standalone Tomcat with a WAR file is the classic deployment, but the game server also runs a TCP listener and a queue that must share state with the servlets; embedding Tomcat keeps one process per tier, one start script per tier, and lets the tests start a real Tomcat on a free port. The cost is that Tomcat's own management features (manager app, hot redeploy) are not used.

H2 versus MySQL.
MySQL is more familiar, but needs an installation and a running service on the demo machine. H2 is a real SQL server started from a jar in the project. The SQL is standard apart from one `MERGE ... KEY` statement, so moving to another database means changing the JDBC URL and that statement.

Asynchronous persistence versus confirmation.
Writes happen after the turn on the persistence thread, so a player is never delayed by the database, but a crash between a match ending and the write completing loses that result. For a game leaderboard that is the right side of the trade.

7.4 Critical analysis of the concurrency mechanisms

Java offers several mechanisms for each problem the system faces. This section weighs them for LUDO-T specifically [1, 2].

Protecting a room's state. Options: `synchronized` methods; a `ReentrantLock`; a `ReentrantReadWriteLock`; immutable state swapped through an `AtomicReference`; or one single-threaded executor per room so that no lock is needed. `synchronized` and `ReentrantLock` give the same exclusion; the explicit lock was chosen because it makes the critical section visible, allows a fairness policy or `tryLock()` later, and provides `Condition`s. A read/write lock would let state readers overlap, but a turn takes microseconds and readers are rare, so the complexity buys nothing measurable. Immutable state would mean rewriting the Assignment 1 model, which mutates pieces in place, and discarding its tests. An executor per room is attractive — it is how Swing works — but needs one thread per room and recreates the unbounded thread growth the queue avoids.

Absorbing bursts. Options: thread per request; an unbounded `LinkedBlockingQueue`; a bounded `ArrayBlockingQueue`; a `SynchronousQueue`; or submitting each request to a `ThreadPoolExecutor`. Thread-per-request has no bound. An unbounded queue never refuses work but can exhaust memory under exactly the sustained flood the rubric describes. A `SynchronousQueue` buffers nothing. A `ThreadPoolExecutor` would work, but the request's session and enqueue time would disappear into a lambda; an explicit `RequestTask` keeps them, which is what makes the wait-time metric and the tests possible. The bounded `ArrayBlockingQueue` with a timed `offer()` parks producers and consumers on two conditions without spinning. Its capacity of 2,000 is under a second of work at the measured processing time.

How many consumers, and which kind of thread. A cached pool grows without limit under load, which is the failure being avoided. Java 21 virtual threads would suit the *blocking socket readers* well and could replace their cached pool with one line; they were not adopted because the readers call `synchronized sendMessage()` blocks, which pin virtual threads in JDK 21, and the connection counts here never justify the audit. For consumers the bound matters more than

the thread type: eight workers give parallelism across rooms and at most eight competitors for any one room's lock.

Serving HTTP: Tomcat's pool versus the JDK server's executor. Both give a fixed pool of request threads. The difference is what happens when a request has to wait. With the JDK server a waiting request holds its thread, so long-polling for events would let 16 idle players exhaust the pool. Tomcat's asynchronous servlets release the thread while waiting and resume on another one, so waiting players cost a little memory, not a thread. Tomcat's `acceptCount` backlog is the HTTP equivalent of the request queue's back-pressure: when all 50 threads are busy, new connections wait in the operating system's queue instead of being refused.

Waking a waiting request. For a roll sent over HTTP the servlet waits briefly for the outcome. Options: busy-waiting, sleep-polling, `wait/notify`, a `Condition`, or a `CompletableFuture`. The first two waste a thread or add latency. A `Condition` pairs with the lock that already guards the session and offers `awaitNanos()` for a clean timeout. A future is neat for single answers, but the room does not know which event completes a request (a roll may end in `YOUR_TURN`, `WAITING_TURN`, `ERROR` or `GAME_OVER`). For long-polling, the waiting is moved out of the thread altogether with `AsyncContext`, and an `AtomicBoolean` compare-and-set guarantees the response is written exactly once even when an event and the timeout race.

Concurrent database writes. Options: a Java lock around every write; one shared JDBC connection; or a connection pool with the database providing isolation. A Java lock works only within one process and serialises reads with writes. A shared `Connection` is not designed for concurrent use. The pool gives each call its own connection and leaves isolation to H2's row locks. The accumulating player-statistics upsert avoids read-modify-write in Java altogether: `UPDATE ... SET games_played = games_played + ?` is atomic in the database, and if the row does not exist yet the `INSERT` runs; if another writer inserted it first, the primary key rejects the second insert and the update is retried. A test drives eight threads × 50 writes of the same new player and loses nothing.

Counters, flags and shared maps. Metrics are incremented by many threads: a `synchronized` counter serialises every enqueue, a `volatile long` is not atomic for increments, and `AtomicLong` uses a non-blocking compare-and-swap. Rooms, sessions and seats live in `ConcurrentHashMaps`: `computeIfAbsent()` makes get-or-create atomic, and iteration is safe while another thread inserts — a plain `HashMap` would be undefined there. `volatile` is enough for run flags written by one thread and read by others.

Client side. The choice was a dedicated reader thread with `invokeLater()`, a `SwingWorker`, or a `javax.swing.Timer` polling the socket. `SwingWorker` is built for tasks that finish; a reader never does, and batching its `publish()` calls would delay events. Polling would need non-blocking I/O and wake the EDT needlessly. The reader thread is the simplest correct design; animations use `javax.swing.Timer`, whose callbacks already run on the EDT.

Table 9: Concurrency mechanisms used and rejected

Mechanism	Used for	Rejected for	Reason
`ReentrantLock` + `finally`	Room state	–	Explicit critical section; no nested locks

Mechanism	Used for	Rejected for	Reason
`synchronized`	Socket `sendMessage()`, chat history	Room state	Fine for tiny leaf sections
`ReentrantReadWriteLock`	–	Room state	Readers rare and short
`ArrayBlockingQueue`, timed `offer`	Request buffer	Unbounded or synchronous queues	Bounded memory, back-pressure
Fixed pool (8)	Queue consumers	Cached pool	Bounded contention
Cached pool	Socket readers	–	Threads idle in I/O; grows only with connections
Virtual threads	–	Socket readers	Pinning audit needed; unnecessary at this scale
Single-thread executor	Persistence writer	Pool	Writes must stay ordered
Tomcat NIO pool + `acceptCount`	HTTP requests	JDK `HttpServer` executor	Bounded pool with OS backlog
`AsyncContext`	Event long-polling	Blocking poll; client-side polling loop	Waiting costs no thread
`Condition.awaitNanos`	Waiting for an action's outcome	Busy-wait, sleep, future	Timeout, multi-outcome
`JdbcConnectionPool` + DB row locks	Database access	Shared connection; Java lock	Isolation where the data is
`AtomicLong`, `AtomicBoolean`	Metrics, exactly-once response	`synchronized`, `volatile`	Lock-free CAS
`ConcurrentHashMap`	Rooms, sessions, seats	`HashMap` under a lock	Safe iteration; atomic get-or-create
`ConcurrentLinkedQueue`	HTTP outbox	Synchronised list	Non-blocking add and drain
`ScheduledExecutorService`	Heartbeat storm, session reaper	–	Periodic tasks without a thread per timer
`invokeLater`, Swing `Timer`	All view updates, animations	`SwingWorker`, polling	EDT rule
`CountDownLatch`	Test choreography	–	Deterministic concurrency tests

7.5 Clashes between patterns and principles

Singleton versus multiple rooms. `Dice.getInstance()` returns one object for the whole JVM, so every room rolls the same dice. It is functionally harmless — `java.util.Random` is thread-safe and rolls are independent — but it couples rooms through the seed. The honest fix is a dice per room, which would mean dropping the Singleton; it was kept because it is part of the preserved Assignment 1 design, and the coupling is documented here.

SRP versus a single critical section. `GameRoom` owns seats, turn flow, prompts and broadcasts. Splitting it would spread the one locked region across classes and make it harder to see that the lock covers everything. Cohesion of the critical section was put first, as `GameEngine`'s size was in Assignment 1.

Information Expert versus the wire. `Piece` knows its own legal movement, but the client has only strings. Rather than copy rule knowledge into the client, the server computes the legal pieces and sends them with `YOUR_TURN`: the expert stays on the server and the client stays thin.

7.6 Limitations and what could be done better

Clients cannot reconnect and resume: the state is saved each round, but not yet used to restore a match. HTTP players long-poll instead of using WebSockets, which Tomcat supports and which would be the natural next step for a browser client. `GameRoom` remains the largest class; extracting a turn coordinator would reduce it. The JMeter and stress runs were made on one machine, so they measure the server's overhead rather than network latency.

8 Test Cases and their Updates

8.1 Test suite

Verification has four layers: the 216 Assignment 1 unit tests, unchanged and still green; 64 tests for everything Assignment 2 added; the automatic stress clients; and the external tools named in the brief, JMeter and Postman. `test.bat` compiles everything and runs the JUnit suite (Figure 15).

Table 10: Test classes for Assignment 2

Test class	Tests	What it proves
`NetworkProtocolTest`	9	Every field round-trips; delimiters, newlines and Unicode are safe; garbage is rejected with `null`, never an exception.
`GameRoomTest`	16	Seating and colour fallback; `ROOM_FULL`; auto-start on the fourth join; out-of-turn and pre-start actions rejected with a reason; moving before rolling rejected; overflow players seated together in `<id>-2` and their match starts; bonus roll on a six; the state snapshot follows the turn; chat reaches every player; a full bot-driven match finishes, spawns the mystery cell and records results.
`RequestQueueProcessorTest`	6	Work runs on worker threads; 16 producers × 200 requests all processed; with every worker blocked and the buffer full, the next producer blocks and nothing queued is lost; 10,000 conflicting requests leave the board consistent; a throwing task does not kill its worker.
`LudoServerIntegrationTest`	5	Real sockets and real embedded Tomcat on free ports: handshake; four socket bots finish a match; two socket and two HTTP players share one room to game over; every endpoint including 400/401/404/405; asynchronous long-poll is held while idle and woken early by an event.

Test class	Tests	What it proves
`JdbcPersistenceGatewayTest`	6	Schema applied and empty; ordering done by SQL; accumulation; awkward characters round-trip; duplicate match rejected loudly; 8 threads × 50 concurrent upserts lose nothing and return every connection.
`DatabaseServerTest`	5	The H2 server creates the schema; data survives a restart; `-reset` restores the initial state; the gateway factory uses the server or falls back; an unreachable server is reported.
`HttpClientSessionTest`	9	Outbox ordering and bound; `awaitEvent` wakes from another thread and ignores stale events; the long-poll listener fires once, fires at once when events are waiting, and never fires after cancellation; `waitMs` is clamped.
`JsonTest`	3	Escaping, nesting, insertion order.
`TurnManagerRulesTest`	5	Three-sixes penalty, blockade break, briefing escape (rules shared by CLI and room).
Assignment 1 suite (20 classes)	216	Domain, strategies, teleports, engine, CLI — unchanged.
Total	**280**	**280 passed, 0 failed**

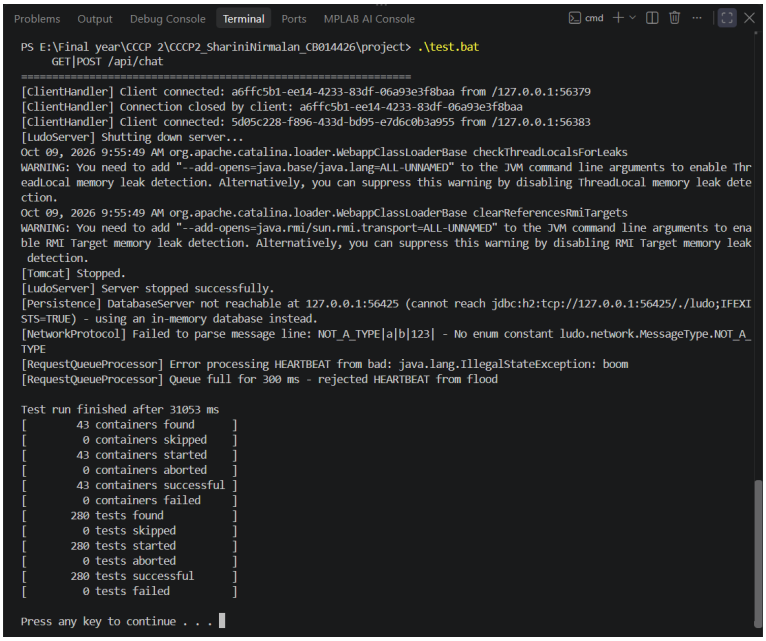


Figure 15: `test.bat`: 280 tests, 0 failures

8.2 How the tests were updated

- **Assignment 1 tests:** untouched. They were the regression net for every refactoring step.
- **Database tests rewritten.** `DatabaseManagerTest` (file store) and `DatabaseServerClientTest` (custom protocol) were deleted with the code they tested and replaced by

`JdbcPersistenceGatewayTest` and `DatabaseServerTest`, which run real SQL against in-memory H2 and against a real H2 server on a free port with a temporary data directory.

- **Servlet tests replaced.** The old `HttpServletTest` tested a hand-written servlet base class through fake request and response objects. With Tomcat that class no longer exists, so its dispatch test was removed; Tomcat's own dispatch is exercised end-to-end by the integration test instead. The remaining session tests, plus new long-poll tests, became `HttpClientSessionTest`.
- **Room and integration tests updated.** Each test now gets its own in-memory database, closed in `@AfterEach`. The integration test waits for Tomcat as well as the TCP port, checks that responses really come from Tomcat, and adds the long-poll test.
- **New tests for defects found by tools.** The overflow, pre-start and move-before-roll tests in `GameRoomTest` pin the fixes for I13 and I14, found by JMeter; a further test pins I17, found by watching the GUI during a live match.
- **Tool artefacts updated.** The JMeter plan's ports became properties (`-JtcpPort`, `-JhttpPort`) and its TCP assertion accepts the greeting line; the Postman collection checks for Tomcat and gained a long-poll request.

8.3 Testing concurrency deterministically

A concurrency test is only useful if it fails when the code is wrong. Three techniques were used. A `CountDownLatch` start gate releases all producer threads at once so they really contend. A `RecordingSession` test double implements `ClientSession` and records every message, so tests can compare what each player saw. And the full-buffer path is forced rather than hoped for: the eight workers are blocked on a latch, the queue is filled to exactly 2,000, and the 2,001st producer is timed.

```
95   void backPressureWhenFull() throws Exception {
96       CountDownLatch release = new CountDownLatch(1);
97       CountDownLatch blockedWorkers = new CountDownLatch(RequestQueueProcessor.WORKER_THREADS);
98       ClientSession blocker = new RecordingSession("blocker") {
99           @Override public void sendMessage(GameMessage message) {
100              blockedWorkers.countDown();
101              try { release.await(); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
102              super.sendMessage(message);
103          }
104      };
105      // occupy every consumer thread
106      for (int i = 0; i < RequestQueueProcessor.WORKER_THREADS; i++) {
107          assertTrue(queue.enqueue(new RequestTask(blocker, new GameMessage(MessageType.HEARTBEAT, "b", "ROOM-1"))));
108      }
109      assertTrue(blockedWorkers.await(2, TimeUnit.SECONDS), "all workers should now be busy");
110
111      // fill the bounded buffer exactly
112      RecordingSession s = new RecordingSession("flood");
113      for (int i = 0; i < RequestQueueProcessor.QUEUE_CAPACITY; i++) {
114          assertTrue(queue.enqueue(new RequestTask(s, new GameMessage(MessageType.HEARTBEAT, "flood", "ROOM-1"))), "slot "
115      }
116      assertEquals(RequestQueueProcessor.QUEUE_CAPACITY, queue.getQueueSize());
117
118      // one more: the producer blocks (back-pressure) and is only refused once the timeout elapses
119      long t0 = System.nanoTime();
120      assertFalse(queue.enqueue(new RequestTask(s, new GameMessage(MessageType.HEARTBEAT, "flood", "ROOM-1")), 300));
121      long blockedMs = TimeUnit.NANOSECONDS.toMillis(System.nanoTime() - t0);
122      assertTrue(blockedMs >= 280, "producer must wait for a slot before giving up, waited " + blockedMs + " ms");
123      assertEquals(1, queue.getTotalDropped());
```

Figure 16: `RequestQueueProcessorTest.backPressureWhenFull()`: forcing the full-buffer path

Because several new tests depend on timing and real servers, the server, servlet and database test classes were run three further times in a row; all 48 passed each time (before the GUI regression test was added).

9 Conclusion

The Assignment 1 domain survived the move to three tiers without modification, which is the clearest evidence that its architecture was sound: networking, a GUI, HTTP and SQL were added as outer layers around it. The concurrency design keeps the number of threads doing game work fixed regardless of load, serialises each room with one explicit lock, never holds that lock while waiting for I/O, and refuses work explicitly rather than silently. Finally, running JMeter and the live GUI against the real system found four defects (I13, I14, I16, I17) that the unit tests had not, which is perhaps the most useful lesson of the assignment.

References

[1] Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D. and Lea, D. (2006) *Java Concurrency in Practice*. Boston: Addison-Wesley.

[2] Lea, D. (2000) *Concurrent Programming in Java: Design Principles and Patterns*. 2nd edn. Boston: Addison-Wesley.

[3] Martin, R.C. (2017) *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall.

[4] Martin, R.C. (2008) *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River: Prentice Hall.

[5] Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading: Addison-Wesley.

[6] Larman, C. (2004) *Applying UML and Patterns*. 3rd edn. Upper Saddle River: Prentice Hall.

[7] Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.

[8] Coffman, E.G., Elphick, M. and Shoshani, A. (1971) 'System deadlocks', *ACM Computing Surveys*, 3(2), pp. 67–78.

[9] McCabe, T.J. (1976) 'A complexity measure', *IEEE Transactions on Software Engineering*, SE-2(4), pp. 308–320.

[10] Eclipse Foundation (2022) *Jakarta Servlet Specification 6.0*, section 'Asynchronous processing'. Available at: `https://jakarta.ee/specifications/servlet/6.0/` (Accessed: 4 October 2026).

[11] Apache Software Foundation (2026) *Apache Tomcat 10.1 Documentation: the HTTP Connector*. Available at: `https://tomcat.apache.org/tomcat-10.1-doc/config/http.html` (Accessed: 4 October 2026).

[12] H2 Database Engine (2026) *Features: client/server mode and concurrency*. Available at: `https://www.h2database.com/html/features.html` (Accessed: 4 October 2026).

[13] Oracle (2023) *Java SE 21 API Specification: java.util.concurrent*. Available at: `https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/package-summary.html` (Accessed: 4 October 2026).

[14] Oracle (2023) *JEP 444: Virtual Threads*. Available at: `https://openjdk.org/jeps/444` (Accessed: 4 October 2026).

[15] Oracle (2023) *The Java Tutorials: Concurrency in Swing*. Available at: `https://docs.oracle.com/javase/tutorial/uiswing/concurrency/` (Accessed: 4 October 2026).

[16] JUnit Team (2024) *JUnit 5 User Guide*. Available at: `https://junit.org/junit5/docs/current/user-guide/` (Accessed: 4 October 2026).

[17] Apache Software Foundation (2024) *Apache JMeter User's Manual*. Available at: `https://jmeter.apache.org/usermanual/` (Accessed: 4 October 2026).

[18] Postman, Inc. (2024) *Newman: command-line collection runner*. Available at: `https://learning.postman.com/docs/collections/using-newman-cli/command-line-integration-with-newman/` (Accessed: 4 October 2026).

A Running the System

Table 11: Launch scripts

Command	Effect
`PLAY_GAME.bat`	Compiles, starts the database server, the game server and four client windows; the match starts when the fourth player joins.
`run_database.bat [port] [-reset]`	Tier 3: H2 server on 8082; `-reset` restores the empty initial state.
`run_server.bat`	Tier 2: TCP 8080, Tomcat 8081, connects to the database on 8082 (in-memory fallback).
`run_client.bat [host port name COLOUR]`	Tier 1: a Swing client (connection dialog without arguments).
`run_stress_test.bat [host port bots seconds]`	Automatic clients against a private or a running server.
`test.bat`	Compiles and runs the 280 JUnit tests.
`jmeter -n -t ludo_jmeter_test_plan.jmx`	JMeter plan; `-JtcpPort`, `-JhttpPort`, `-Jhost` override the defaults.
`newman run Ludo_Postman_Collection.json`	The Postman collection from the command line.

B HTTP API (embedded Tomcat, port 8081)

Table 12: HTTP endpoints

Endpoint	Purpose
`GET /api/room/status?roomId=`	Room summary and seats.
`POST /api/room/join`	`roomId`, `playerName`, `preferredColor`; returns a `clientId`.
`POST /api/room/leave`	`clientId`.
`POST /api/game/roll`	`clientId`; returns the dice value, or 409 when it is not the player's turn.
`POST /api/game/move`	`clientId`, `piece`.
`GET /api/game/events? clientId=&waitMs=`	Drains the player's events; with `waitMs` (up to 30,000) it long-polls asynchronously.
`GET /api/game/state?roomId=`	Full board snapshot.
`GET|POST /api/chat`	Read or post chat.
`GET /api/leaderboard`	Rankings and match history from the H2 database.
`GET /api/stats`	Queue counters, rooms, JVM figures.